Name: Bonita, Kier Nathaniel M.	**Date Performed: 03/05/2026**
Course/Section: CPE212 - CPE31S1	**Date Submitted: 03/05/2026**
Instructor: Engr. Robin Valenzuela	**Semester and SY: 2nd - 2025-2026**

Activity 8: Install, Configure, and Manage Availability Monitoring tools

1. Objectives

Create and design a workflow that installs, configure and manage enterprise monitoring tools using Ansible as an Infrastructure as Code (IaC) tool.

2. Discussion

Availability monitoring is a type of monitoring tool that we use if the certain workload is up or reachable on our end. Site downtime can lead to loss of revenue, reputational damage and severe distress. Availability monitoring prevents adverse situations by checking the uptime of infrastructure components such as servers and apps and notifying the webmaster of problems before they impact on business.

3. Tasks

1. Create a playbook that installs Nagios in both Ubuntu and CentOS. Apply the concept of creating roles.
2. Describe how you did step 1. (Provide screenshots and explanations in your report. Make your report detailed such that it will look like a manual.)
3. Show an output of the installed Nagios for both Ubuntu and CentOS.
4. Make sure to create a new repository in GitHub for this activity.

4. Output (screenshots and explanations)

I created a new directory called nagios, navigated into it, then used the mkdir -p command to manually set up the folder structure roles/nagios/tasks and roles/nagios/handlers, which prepares the standard role layout used for organizing configuration tasks. After that, I used the touch command to create two empty files which are site.yml and roles/nagios/tasks/playbook.yml which I removed later on because it wasn't correct

```
kier@LocalMachine:~$ mkdir nagios
kier@LocalMachine:~$ cd nagios
kier@LocalMachine:~/nagios$ mkdir -p roles/nagios/tasks
kier@LocalMachine:~/nagios$ mkdir -p roles/nagios/handlers
kier@LocalMachine:~/nagios$ touch site.yml
kier@LocalMachine:~/nagios$ touch roles/nagios/tasks/playbook.yml
```

```
kier@LocalMachine:~/nagios$ tree
.
├── ansible.cfg
├── inventory.ini
├── roles
│   └── nagios
│       ├── handlers
│       └── tasks
│           └── main.yml
└── site.yml
```

Here I assigned inventory.ini as inventory which has my ip addresses for my ubuntu and centOS that I will be using later on

```
main.yml   ansible.cfg ×   inventory.ini   site.yml
ansible.cfg
1  [defaults]
2  inventory = inventory.ini
3  private_key_file = /home/kier/.ssh/id_ed25519
```

Here in inventory.ini, as I've said earlier, I have my ip addressed inside this file.

```
main.yml   ansible.cfg   inventory.ini ×   site.yml
inventory.ini
1  [ubuntu]
2  server1 ansible_host=192.168.56.101
3  [centos]
4  server2 ansible_host=192.168.56.114
```

Here I created a main Ansible playbook file (site.yml) that defines the automation task for installing Nagios on the target machines. I specified hosts: all to apply the playbook to every host listed in the inventory, enabled administrative privileges using become: true, and included the nagios role so that Ansible will execute the tasks defined inside the roles/nagios directory to perform the Nagios installation and configuration.

```yaml
---
- name: Install Nagios
  hosts: all
  become: true

  roles:
    - nagios
```

Here I created an Ansible task file (main.yml) that installs and manages the Nagios service on different Linux distributions by using conditional statements based on the system's ansible_os_family. In this configuration, I used the dnf module to install the EPEL repository and Nagios on RedHat/CentOS systems, and the apt module to install nagios4 on Debian/Ubuntu systems, then ensured the appropriate Nagios service is enabled and started using the service module depending on the detected operating system.

main.yml | ansible.cfg | inventory.ini | site.yml

roles > nagios > tasks > main.yml > { } 4 > when

Ansible Tasks Schema - Ansible tasks file (ansible.json)

```yaml
---

- name: Install EPEL repository (CentOS)
  dnf:
    name: epel-release
    state: present
  when: ansible_os_family == "RedHat"

- name: Install Nagios on Ubuntu
  apt:
    name: nagios4
    state: present
    update_cache: yes
  when: ansible_os_family == "Debian"

- name: Install Nagios on CentOS
  dnf:
    name: nagios
    state: present
  when: ansible_os_family == "RedHat"

- name: Enable and Start Nagios on Ubuntu
  service:
    name: nagios4
    state: started
    enabled: yes
  when: ansible_os_family == "Debian"

- name: Enable and Start Nagios on CentOS
  service:
    name: nagios
    state: started
    enabled: yes
  when: ansible_os_family == "RedHat"
```

These are the proofs that I have successfully installed, enabled, and running nagios on both Ubuntu and CentOS.

```
kier@server1:~/Desktop$ systemctl status nagios4
● nagios4.service - nagios4
     Loaded: loaded (/usr/lib/systemd/system/nagios4.service; enabled; preset: >
     Active: active (running) since Thu 2026-03-05 17:53:35 PST; 3min 29s ago
       Docs: man:nagios4
   Main PID: 8602 (nagios4)
      Tasks: 8 (limit: 11550)
     Memory: 2.9M (peak: 4.2M)
        CPU: 58ms
     CGroup: /system.slice/nagios4.service
             ├─8602 /usr/sbin/nagios4 /etc/nagios4/nagios.cfg
             ├─8604 /usr/sbin/nagios4 --worker /var/lib/nagios4/rw/nagios.qh
             ├─8605 /usr/sbin/nagios4 --worker /var/lib/nagios4/rw/nagios.qh
             ├─8606 /usr/sbin/nagios4 --worker /var/lib/nagios4/rw/nagios.qh
             ├─8607 /usr/sbin/nagios4 --worker /var/lib/nagios4/rw/nagios.qh
             ├─8608 /usr/sbin/nagios4 --worker /var/lib/nagios4/rw/nagios.qh
             ├─8609 /usr/sbin/nagios4 --worker /var/lib/nagios4/rw/nagios.qh
             └─8817 /usr/sbin/nagios4 /etc/nagios4/nagios.cfg
```

```
[kier@vbox ~]$ systemctl status nagios
● nagios.service - Nagios Core 4.4.14
     Loaded: loaded (/usr/lib/systemd/system/nagios.service; enabled; preset: d>
     Active: active (running) since Thu 2026-03-05 17:56:43 PST; 48s ago
       Docs: https://www.nagios.org/documentation
    Process: 14140 ExecStartPre=/usr/sbin/nagios -v /etc/nagios/nagios.cfg (cod>
    Process: 14177 ExecStart=/usr/sbin/nagios -d /etc/nagios/nagios.cfg (code=e>
   Main PID: 14190 (nagios)
      Tasks: 8 (limit: 60072)
     Memory: 7.4M
        CPU: 226ms
     CGroup: /system.slice/nagios.service
             ├─14190 /usr/sbin/nagios -d /etc/nagios/nagios.cfg
             ├─14192 /usr/sbin/nagios --worker /var/spool/nagios/cmd/nagios.qh
             ├─14193 /usr/sbin/nagios --worker /var/spool/nagios/cmd/nagios.qh
             ├─14194 /usr/sbin/nagios --worker /var/spool/nagios/cmd/nagios.qh
             ├─14195 /usr/sbin/nagios --worker /var/spool/nagios/cmd/nagios.qh
             ├─14196 /usr/sbin/nagios --worker /var/spool/nagios/cmd/nagios.qh
             ├─14197 /usr/sbin/nagios --worker /var/spool/nagios/cmd/nagios.qh
             └─17524 /usr/sbin/nagios -d /etc/nagios/nagios.cfg
```

GitHub Link

Reflections:

Answer the following:

1. What are the benefits of having an availability monitoring tool?
   - An availability monitoring tool helps monitor the status of servers, services, and network devices to ensure they are running properly. It also helps detect issues early and allows administrators to respond quickly to reduce downtime and maintain system reliability.

Conclusions:

In this activity, I successfully configured an Ansible environment to automate the installation and management of Nagios on both Ubuntu and CentOS systems. By creating the proper directory structure, inventory file, playbook, and task files with conditional configurations, I was able to deploy Nagios and ensure that the service was properly installed, enabled, and running on both target machines.